Dhirubhai Ambani University (DAU)

Department of Information and Communication Technology

IT 305: Computer Networks Lab — Autumn 2026–2027

Socket Programming Project: Design Document

Fault Tolerant Topic-based File Distribution Service

Project Group: 17 **Lab Groups:** 021 and 034 (merged)

Platform: C11 on Linux, POSIX sockets (IPv4, TCP), POSIX threads, GNU Make

Sr.	Student Name	Roll No.	Planned Responsibility
1	Aaditya Kaushik (Group 034)	202401001	Part I server: topic map, single and multi-threaded modes
2	Nisarg Trivedi (Group 034)	202401129	Part I client, buffered line reader, metrics logging
3	Niranjan Dangi (Group 034)	202401042	Fault injection and Part II Case 1 retry logic
4	Kavish Inani (Group 021)	202401089	Part II Case 2: session table and resume protocol
5	Neel Khatri (Group 021)	202401093	Case 2 Enhanced: parallel connections and optional optimizations
6	Manal Patel (Group 021)	202401112	Experiment scripts, plots, report integration

Contents

1 System Overview 2

2 Server Data Structures: Topic to Directory Map 2

3 Client-Server Message Formats 4

4 Fault-Injection Mechanism 6

5 Session Management Protocol (Case 2) 8

6 Performance Enhancements (Case 2 Enhanced) 10

7 Additional Features Beyond the Minimum 12

8 Code Structure 12

9 Design Thought Process and Alternatives 14

1 System Overview

The service is engineered as one unified C codebase structured into four cumulative layers: a basic topic-based file server (**Part I**), fault injection with restart semantics (**Case 1**), resumable checkpointed sessions (**Case 2**), and parallel TCP connections with dynamic work queue distribution (**Case 2 Enhanced**). Each subsequent layer directly reuses and builds upon the code of preceding layers, ensuring that any observable variance in operational behavior between cases is attributed to exactly one added mechanism.

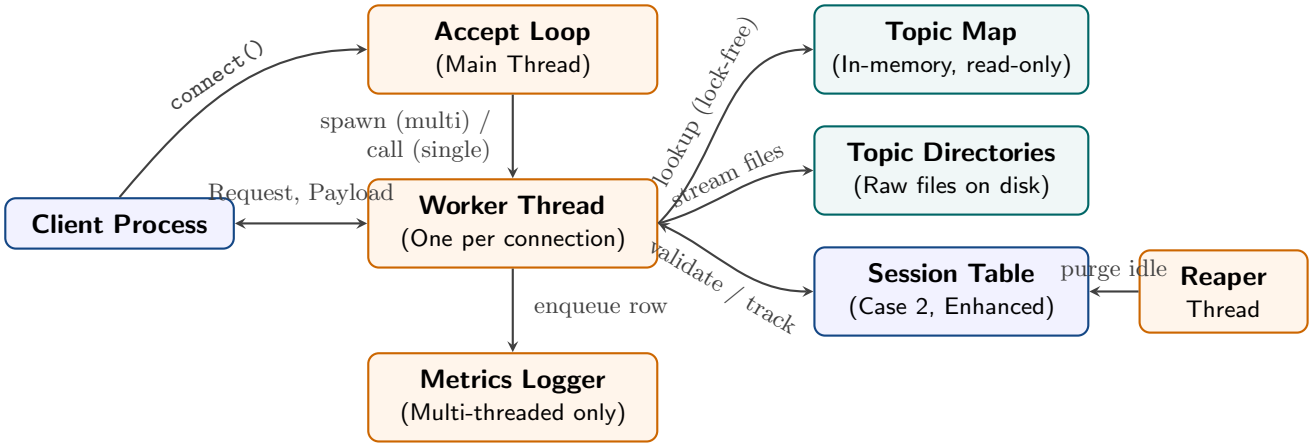


Figure 1: Server architecture and thread interactions. The main thread executes the accept loop while worker threads handle all per-client socket I/O. The topic map is statically initialized once at startup and read concurrently without locks. The session table forms the sole shared mutable synchronization state in Part II.

In Part I single-threaded mode, the server process maintains strictly one thread of execution. That thread accepts an incoming client connection, serves all files belonging to the requested topic sequentially until completion, and only then proceeds to accept the next queued connection. No asynchronous logger or reaper thread is instantiated in this mode; the serving thread directly appends each performance log row to disk only after the client's end-of-transfer timestamp has been recorded, guaranteeing that logging overhead never distorts measured network transfer intervals. The dedicated logger thread is active exclusively in multi-threaded configurations, and the background session reaper thread operates only in Case 2 and Case 2 Enhanced.

2 Server Data Structures: Topic to Directory Map

The server scans `<topics_root_dir>` exactly once during initial startup and constructs a dynamically allocated, sorted in-memory array of topics. Each topic structure contains a sorted array of file metadata descriptors. Topic lookups are performed in $\mathcal{O}(\log m)$ time using standard POSIX `bsearch()`. No topic keys or directory paths are hardcoded: every valid sub-folder located directly inside the root directory is recognized as an independent topic, with its folder name serving as the topic key.

```
typedef struct {
    char     name[256];      /* File name in topic folder */
    uint64_t size;           /* Size in bytes from stat() */
    uint64_t pos;            /* Cumulative payload offset in topic */
} file_entry_t;

typedef struct {
    char          key[64];          /* Topic identifier (e.g. "cane") */
    char          path[PATH_MAX];   /* Absolute path from realpath() */
    file_entry_t *files;            /* Array sorted alphabetically */
    uint32_t      file_count;       /* Number of regular files */
    uint64_t      total_bytes;      /* Total payload bytes */
} topic_t;
```

```c
typedef struct {
    topic_t *topics;          /* Sorted array; read-only */
    uint32_t count;           /* Total active topics */
} topic_map_t;
```

Listing 1: Topic mapping data structures.

When initialized against the reference Animals-10 dataset, the topic table automatically resolves the mappings summarized in Table 1. The root prefix represents the arbitrary directory path supplied on the server command line.

Table 1: Topic map resolution for the Animals-10 dataset.

Key (Topic Identifier)	Resolved Value (Absolute Directory Path)
cane	<topics_root_dir>/cane
cavallo	<topics_root_dir>/cavallo
elefante	<topics_root_dir>/elefante
farfalla	<topics_root_dir>/farfalla
gallina	<topics_root_dir>/gallina
gatto	<topics_root_dir>/gatto
mucca	<topics_root_dir>/mucca
pecora	<topics_root_dir>/pecora
ragno	<topics_root_dir>/ragno
scoiattolo	<topics_root_dir>/scoiattolo

2.1 Design Notes

- **Lock-Free Concurrent Reads:** The topic map is completely immutable once startup initialization completes. Worker threads read topic metadata concurrently without acquiring any mutex locks, entirely eliminating synchronization contention on metadata lookups in the multi-threaded server. Pointer references to a `topic_t` remain valid for the entire server lifespan.
- **Deterministic File Indexing:** Files within each topic directory are sorted alphabetically by filename (`qsort()` using `strcmp()`). Consequently, every file possesses a deterministic, permanent index in the range $[0, n-1]$, where n is `file_count`. All subsequent checkpoint records, resume requests, and parallel work batches reference files using this stable index.
- **Constant-Time Offset Translation:** The precomputed `pos` field translates any tuple of (file_index, offset) into a single continuous stream offset within the topic in $\mathcal{O}(1)$ time. This also enables the fault-injection subsystem to compute the exact total response payload size R in constant time prior to initiating transfers.
- **Directory Immutability Assumption:** Topic directory trees are assumed to remain static while the server is active. Adding a new topic requires creating a directory under `topics_root_dir`, which is indexed upon the next server launch.

2.2 Lookup and Validation Rules

1. **Case-Insensitive Resolution:** Matching performs an exact string comparison first, followed by case-insensitive matching (`strcasecmp()`). Thus, requests such as `GET Cane` and `GET cane` resolve identically.
2. **Input Sanitization:** Topic keys that are empty, exceed 63 characters in length, or contain directory traversal tokens (`/` or `..`) are rejected immediately with `ERR 400 BAD_REQUEST`.
3. **File Filtering:** Directory scanning checks `stat()` and accepts only regular files (`S_ISREG`). Hidden files (prefix `.`), nested subdirectories, and files containing newline characters in their names are excluded, ensuring every header fits on a single ASCII line.

4. **Lookup Complexity:** For the 10 reference topics, binary search requires at most $\lceil \log_2 10 \rceil = 4$ comparisons. A hash table would introduce unnecessary overhead and code complexity without measurable gain. The lookup is modularized behind `topic_lookup()` to permit drop-in algorithmic replacement if scaling to thousands of topics is needed.

3 Client-Server Message Formats

All protocol control messages are formatted as single ASCII text lines terminated by a single newline character (`\n`). File contents immediately follow their preceding file header as raw binary bytes whose exact byte length is explicitly specified by the header. Text-based control framing preserves human readability, conforms strictly to the assignment's `GET <topic>` specification, and allows effortless inspection via tools such as `netcat`, `curl`, and Wireshark. Since file headers average $\sim$ 50 bytes, control overhead constitutes less than 0.1% for typical multi-kilobyte images.

3.1 Common Framing and Parsing Rules

- **Delimiters and Encoding:** Tokens on control lines are separated by a single ASCII space. Numerical values are encoded as unsigned decimal integers. Maximum control line length is enforced at 1024 bytes (including `\n`).
- **Filename Handling:** In control frames containing a file name, the name is placed as the final token and extends to the trailing newline, safely supporting file names with internal whitespace.
- **Error Responses:** Protocol errors follow the format `ERR <code> <REASON>\n`, after which the server terminates the socket connection.
- **Buffered Reading:** The client implements a robust line reader (`lr_readline`) backed by an internal buffer. Binary file payloads are fetched via `lr_read_exact`, which first drains any remaining bytes in the line reader buffer before issuing raw `recv()` calls, preventing framing truncation when a text header and binary file payload arrive in the same TCP segment.

3.2 Part I: Basic Topic File Distribution

In Part I, the client initiates the transfer with a standard request, receives the topic summary, downloads each file sequentially, and validates aggregate transfer bounds upon receiving the closing delimiter.

Part I Wire Exchange Example

```
C: GET cane\n
S: OK 4863 125829120\n
S: FILE 25600 dog_0001.jpg\n
S: <25600 raw bytes>
... [4863 file blocks transmitted sequentially] ...
S: END 4863 125829120\n
```

Table 2: Part I protocol messages.

Direction	Message Format	Functional Meaning
$C \to S$	`GET <topic>`	Requests full transmission of all files in topic.
$S \to C$	`OK <file_count> <total_bytes>`	Topic verified; reports total files and payload bytes.
$S \to C$	`ERR 404 NO_SUCH_TOPIC` `ERR 400 BAD_REQUEST`	Lookup failed or invalid request syntax; server closes.
$S \to C$	`FILE <size> <name>` followed by `<size>` raw bytes	File descriptor followed immediately by raw payload; repeated `file_count` times.
$S \to C$	`END <file_count> <total_bytes>`	Signals end of topic stream; server closes connection.

The client creates the directory path `<output_dir>/<topic>/`, stores incoming files inside, and terminates with exit code `0` only when `END` is parsed and matches the expected counts from `OK`. Premature EOF before `END` triggers an immediate exit with code `2`.

3.3 Part II Case 1: Fault Injection Without Session Management

The wire protocol and message syntax in Case 1 are strictly identical to Part I; neither client nor server stores persistent session state. Operational differences are entirely behavioral:

- The server probabilistically injects connection aborts at an arbitrary byte index during transmission (Section 4).
- Upon encountering a premature EOF or `ECONNRESET` prior to receiving `END`, the client registers a failure. It discards and unlinks all partially downloaded files from the current attempt, executes exponential backoff starting at 100 ms and doubling up to 2 s, establishes a fresh TCP socket, and re-issues `GET <topic>` from byte 0.
- The client retries up to `max_retries` attempts (default 50). Protocol `ERR` responses are terminal and never retried.

3.4 Part II Case 2: Fault Tolerance With Resumable Sessions

Case 2 introduces explicit session tokens and checkpoint negotiation. When a connection drops, the client resumes exactly from its last durable disk checkpoint.

Table 3: Part II Case 2 protocol messages.

Direction	Message Format	Functional Meaning
$C \to S$	`GET <topic>`	Initiates a new resumable transfer session.
$S \to C$	`SESSION <sid> <file_count> <total_bytes>`	Allocates a 128-bit hex session ID (`sid`) and declares topic totals.
$C \to S$	`RESUME <sid> <topic> <file_index> <offset>`	Requests resumption from client's last disk checkpoint.
$S \to C$	`RESUMED <file_index> <offset> <remaining_bytes>`	Server validates checkpoint; resumes data stream.

$S \to C$	`ERR 410 SESSION_EXPIRED`
	`ERR 409 TOPIC_MISMATCH`
	`ERR 416 BAD_CHECKPOINT`
		Resumption rejected; client falls back to fresh `GET`.

$S \to C$	`FILE <index> <size> <start> <name>` followed by (`size` − `start`) raw bytes	File header indicating file index, total size, starting byte offset, and filename.
$S \to C$	`END <file_count> <total_bytes>`	All topic data delivered.
$C \to S$	`DONE <sid>`	Client confirms all files flushed to disk.
$S \to C$	`BYE`	Server releases session state and terminates socket.

Operational Semantics:

- The `<start>` parameter is `0` for all whole files, and equals `offset` for the first file transmitted following a `RESUMED` response.
- A checkpoint of (`file_index` = `file_count`, `offset` = 0) signifies that all files are already completely stored on disk and only the trailing `END` delimiter was dropped. The server responds with `RESUMED <file_count> 0 0` followed immediately by `END`.
- The client treats arrival of `END` as transfer success. The `DONE` / `BYE` exchange serves as best-effort session cleanup: if the connection drops prior to `BYE`, the transfer remains successfully completed, and the server reaper thread reclaims the orphaned session upon TTL expiration.

3.5 Part II Case 2 Enhanced: Parallel TCP Connections

In the enhanced variant, Connection 1 negotiates the session and retrieves a full file manifest. Up to N parallel TCP connections (default 4) then attach to the active session and cooperatively pull dynamic file batches using `FETCH`.

Table 4: Part II Case 2 Enhanced protocol messages.

Direction	Message Format	Functional Meaning
$C \to S$ (Conn 1)	`GET <topic> <n_conns>`	Initiates parallel session and declares planned worker count N.
$S \to C$	`SESSION <sid> <file_count> <total_bytes>`	Returns session token and totals (identical to Case 2).
$S \to C$	`M <index> <size> <name>` (repeated) `MEND`	Streams topic manifest entries followed by manifest terminator.
$C \to S$ (Conns 2..N)	`ATTACH <sid>`	Worker connections join active session.
$S \to C$	`ATTACHED <sid>` `ERR 410 SESSION_EXPIRED`	Server confirms worker association or rejects invalid session.
$C \to S$	`FETCH <sid> <first_idx> <last_idx> <offset>`	Worker requests batch of files from `first_idx` to `last_idx`.
$S \to C$	`FILE <index> <size> <start> <name>` followed by payload bytes	Transmits files within requested batch.
$S \to C$	`BEND <first_idx> <last_idx>`	Marks completion of batch; connection kept alive for next `FETCH`.
$C \to S$	`DONE <sid>`	Transmitted over primary connection once all batches complete.
$S \to C$	`BYE`	Server acknowledges and terminates session.

Upon receiving `MEND`, Connection 1 joins the worker pool and pulls batches via `FETCH` identically to attached connections. If a fault disconnects the primary link during manifest transfer, the client simply restarts session negotiation with `GET`.

4 Fault-Injection Mechanism

4.1 Definition of Failure Probability p

For each discrete transfer request received by the server (`GET` or `RESUME`; in Case 2 Enhanced, each new TCP connection), the server evaluates exactly one Bernoulli trial with failure probability $p \in [0, 1]$, supplied via the command-line argument `<failure probability>`. If the trial determines a failure:

1. A cut-point byte offset K is drawn uniformly from the total response bytes R scheduled for transmission:
$$K \sim \mathcal{U}\{0, 1, \ldots, R-1\} \tag{1}$$
2. The worker thread streams data normally until exactly K cumulative bytes (headers plus payload) have been transferred across the socket.
3. At byte K, the socket is immediately terminated using an ungraceful TCP reset.

Thus, p defines the probability that an individual client request experiences an abrupt disconnection. It is an independent per-request trial, rather than a per-byte or per-chunk bit-error rate. Each subsequent reconnection constitutes a distinct request with its own independent trial.

4.2 Step-by-Step Server Fault Decision Flow

1. **Response Sizing:** The worker parses the client request and computes the exact response length R (including all protocol headers and file payloads) in $\mathcal{O}(1)$ time using the precomputed file offsets in `topic_t`.

2. **Random Variate Generation:** The worker draws a uniform floating-point value $u \sim \mathcal{U}[0, 1)$ using POSIX `erand48()`. The random state is maintained on a per-thread basis and seeded via seed $\oplus$ connection_id, guaranteeing reproducibility across experimental trials.

3. **Threshold Comparison:** If $u < p$, a cut offset is selected as $K = \lfloor u' \cdot R \rfloor$ where $u' \sim \mathcal{U}[0, 1)$. If $u \geq p$, $K = R$ (no fault injected).

4. **Enforced Socket Abort:** All socket writes (`send()` or `sendfile()`) are capped so total transmitted bytes never exceed K. When K bytes are reached, the thread enables `SO_LINGER` with a timeout of 0 seconds and invokes `close()`:

```
struct linger sl = { .l_onoff = 1, .l_linger = 0 };
setsockopt(fd, SOL_SOCKET, SO_LINGER, &sl, sizeof(sl));
close(fd);
```

   This sequence transmits a hardware-level `TCP RST` packet, simulating a network crash or ungraceful host drop without entering the graceful `TCP FIN` teardown.

5. **Fault Accounting:** The worker logs `FAULT conn=<id> topic=<t> cut_at=<K> of=<R>` to standard error and terminates cleanly. In Case 2, the session descriptor is preserved in the session table awaiting client reconnection.

Because K is uniformly distributed over the entire response, failures land with realistic diversity across header boundaries, inside binary image data, or between files. Both endpoints set `signal(SIGPIPE, SIG_IGN)` and invoke `send()` with `MSG_NOSIGNAL`, preventing fatal signals on aborted writes.

4.3 Mathematical Analysis: Per-Request vs. Per-Chunk Modeling

A naive fault-injection design evaluates a coin toss with probability q on every transmitted 64 KiB chunk. For a topic of size S bytes split into $n = \lceil S/64\,\text{KiB} \rceil$ chunks, the probability that an entire topic finishes without any failure is:

$$P(\text{completion}) = (1 - q)^n \tag{2}$$

For a moderate 100 MiB topic ($n = 1600$), even a minuscule error rate of $q = 0.01$ yields:

$$P(\text{completion}) = (1 - 0.01)^{1600} \approx 1.0 \times 10^{-7} \tag{3}$$

Under such conditions, Case 1 (restart from zero) would practically never finish.

In contrast, our per-request formulation ensures that completion remains tractable and yields rigorous closed-form expectations. The number of attempts required to achieve success follows a geometric distribution with mean:

$$\mathbb{E}[\text{attempts}] = \frac{1}{1 - p} \tag{4}$$

Since each failed attempt transmits K bytes with expected value $\mathbb{E}[K] = \frac{S-1}{2}$, the expected total redundant bytes transmitted in Case 1 before completion is:

$$\mathbb{E}[\text{bytes transmitted in failed attempts}] = \left(\frac{p}{1 - p}\right) \frac{S - 1}{2} \approx \frac{p}{1 - p} \cdot \frac{S}{2} \tag{5}$$

For Case 2, while the expected number of connection attempts remains $\frac{1}{1-p}$ (since only an uninterrupted run delivers `END`), session checkpoints bound the payload discarded per cut to at most the checkpoint interval c (default 64 KiB):

$$\mathbb{E}[\text{redundant payload bytes}] < \frac{p}{1 - p} \cdot c \tag{6}$$

This shows that session management can reduce the amount of redundant data after a failure from potentially the full topic size to approximately the checkpoint interval.

4.4 Boundary Conditions

- $p = 0$**:** Deterministic baseline execution. No faults are injected; transfers proceed identically to Part I.
- $p = 1$**:** Every request is guaranteed to be cut before delivering `END` ($K \leq R - 1$). Both Case 1 and Case 2 exhaust `max_retries` and exit with code 2. For rigorous empirical evaluation, experimental testing restricts $p \leq 0.9$.

5 Session Management Protocol (Case 2)

The design establishes the client's local disk checkpoint as the definitive authority on transfer progress. The server maintains matching session metadata to authenticate checkpoints and account for network throughput.

5.1 Protocol Message Sequence

Figure 2 illustrates the message sequence during a Case 2 transfer where a fault abort occurs during file 1 after the client has checkpointed the first 64 KiB.

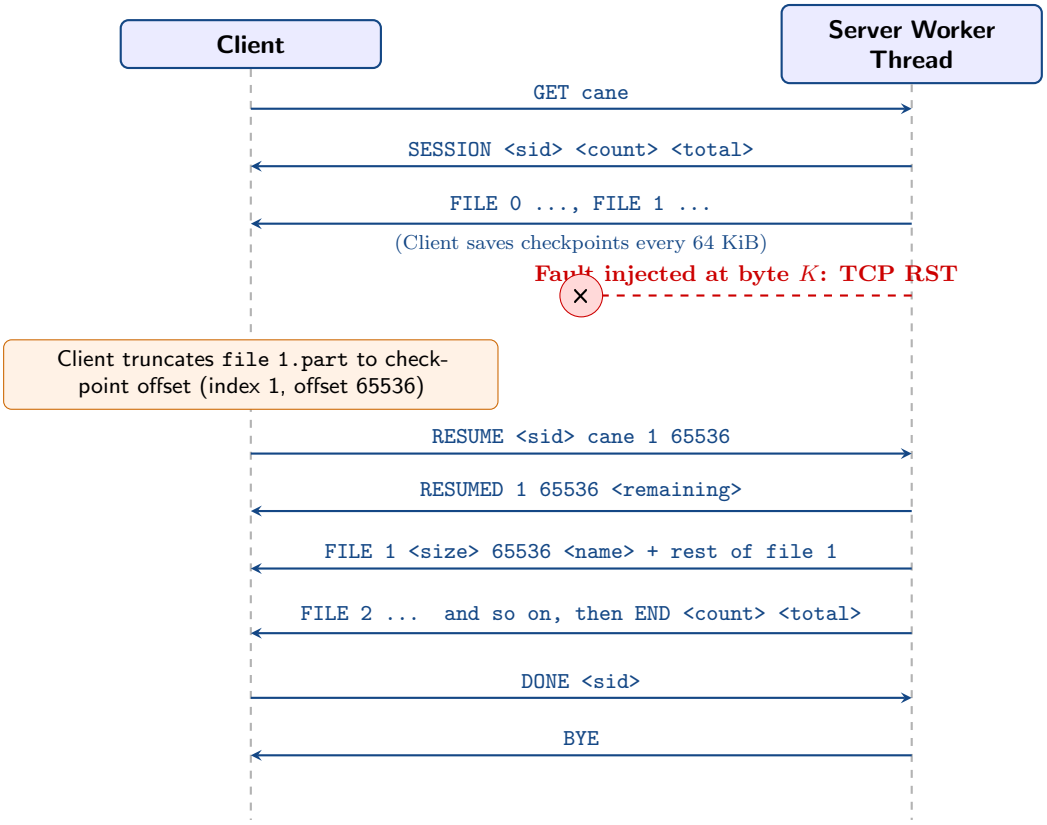


Figure 2: Case 2 session management sequence: transfer interruption, state rollback, checkpoint negotiation, and resumed streaming.

If a resume request is rejected (`ERR 409`, `410`, or `416`), the client discards all temporary files and checkpoints, gracefully falling back to a clean `GET` transfer.

5.2 State Maintained by the Client

Table 5 details the persistent and in-memory state elements tracked by the client process.
Checkpoint Atomicity and Disk Safety: To guarantee that a checkpoint never indexes uncommitted disk data:

Table 5: Client-side state architecture in Case 2.

State Item	Storage Location	Persistence and Update Timing
Checkpoint Record `v1 <sid> <topic> <idx> <off> <done>`	`<output_dir>/<topic>/.ckpt`	Updated after each complete file and every $c = 64$ KiB within a partial file.
Partial File Payload	`<output_dir>/<topic>/<name>.part`	Streamed continuously; atomically renamed to `<name>` upon file completion.
Attempt Counters (attempts, bytes, redundant)	Process memory → Client metrics CSV	Tracked continuously in memory; flushed to CSV upon completion of transfer.

1. Incoming socket bytes are written to `<name>.part` and committed via `fflush()`.
2. Checkpoint state is written to a temporary file `.ckpt.tmp`.
3. The client invokes POSIX `rename(".ckpt.tmp", ".ckpt")`, which provides atomic metadata updates.

Upon reconnection following a crash, the client opens `.ckpt`, parses its last confirmed offset, and invokes `ftruncate()` on `<name>.part` to trim any uncheckpointed trailing bytes, safely discarding redundant bytes. Once the closing `END` delimiter is validated, `.ckpt` is deleted, leaving only completed topic files.

5.3 State Maintained by the Server

Server sessions are managed in an in-memory hash table composed of 256 buckets, keyed by the 32-character hexadecimal session identifier `sid`.

```
typedef struct session {
    char             sid[33];          /* 128-bit hex ID from /dev/urandom */
    const topic_t  *topic;             /* Pointer to immutable topic metadata */
    pthread_mutex_t mu;                /* Mutex guarding progress fields below */
    uint64_t         hw_pos;           /* High-water mark in payload stream */
    uint64_t         bytes_sent;       /* Cumulative bytes across attempts */
    uint32_t         attempts;         /* Total connection attempts */
    uint32_t         generation;       /* Incremented on each RESUME */
    uint32_t         refs;             /* Active connection count (Enhanced) */
    time_t           last_seen;        /* Timestamp of most recent activity */
    struct session *next;              /* Hash bucket collision chain */
} session_t;
```

Listing 2: Server session control block.

Concurrency and High-Water Mark Accounting:

- A global table lock guards bucket lookups, allocations, and deletions. Each `session_t` maintains its own mutex `mu` protecting progress counters.
- The worker thread advances `hw_pos` after every `send()` or `sendfile()` invocation. Because a TCP RST discards bytes lingering in the kernel send buffer, `hw_pos` acts as a safe upper bound: the client could not possibly have received bytes beyond `hw_pos`.
- A dedicated background `reaper_thread` periodically inspects the session table and purges unreferenced sessions that have remained idle for longer than 300 s (`session_ttl`).

5.4 Checkpoint Validation on RESUME

Let n denote the total file count of topic T, and let s_j denote the size of file j. Any checkpoint tuple (i, o), indicating offset o within file i, maps to a global stream payload position:

$$\text{pos}(i, o) = \sum_{j<i} s_j + o = \text{topic.files}[i].\text{pos} + o \tag{7}$$

For $i = n$ (all files complete), $\text{pos}(n, 0) = \text{topic.total_bytes}$. Comparing continuous scalar positions rather than (i, o) tuples correctly eliminates boundary ambiguities between file i at offset s_i and file $i + 1$ at offset 0.

The server accepts `RESUME <sid> <topic> <file_index> <offset>` if and only if:

1. The session ID `sid` exists in the session table and has not expired (else `ERR 410 SESSION_EXPIRED`).
2. The requested `topic` matches the session's bound topic (else `ERR 409 TOPIC_MISMATCH`).
3. Index and offset bounds are valid ($i < n$ and $o \leq s_i$, or $i = n$ and $o = 0$), and $\text{pos}(i, o) \leq \text{hw_pos}$ (else `ERR 416 BAD_CHECKPOINT`).

Upon acceptance, the server increments `generation`. If an orphaned thread from an earlier interrupted connection is still blocked, it detects the incremented generation value upon wake-up and immediately exits, preventing concurrent transmissions on the same session.

6 Performance Enhancements (Case 2 Enhanced)

The mandatory performance enhancement implements N concurrent TCP connections per topic (default $N = 4$, configurable via `conns=<n>`) coordinated by a dynamic client-side batch queue. Optional optimizations detailed in Section 6.2 can be selectively enabled for comparative benchmarking.

6.1 Parallel Transfer Architecture

Table 6 outlines the core architectural components of the parallel transfer engine.

Per-Connection Fault Injection in Enhanced Mode: In Case 2 Enhanced, evaluating fault injection per `FETCH` would expose a 100 MiB topic to ≈ 100 fault trials, compared to 1 trial in Case 2. To ensure fair performance evaluation at identical p values, fault trials are decided **once per TCP connection** (at initial `GET` or `ATTACH`). When a trial selects failure, the abort byte K is drawn over that connection's estimated share of remaining payload:

$$K \sim \mathcal{U}\left\{0, 1, \ldots, \left\lceil \frac{R_{\text{rem}}}{N} \right\rceil - 1\right\} \tag{8}$$

where R_{rem} denotes remaining uncommitted topic payload bytes.

6.2 Optional Performance Enhancements

6.3 Socket Communication Mode

The server architecture uses standard **blocking sockets with dedicated worker threads** (one thread per active connection). With experimental workloads peaking around 64 concurrent connections, blocking I/O provides straightforward linear control flow and transparent error propagation during network faults. Non-blocking I/O using `epoll` introduces state-machine complexity without throughput benefit at this connection scale.

6.4 Ablation Methodology

To rigorously quantify performance benefits, each optimization is toggleable via command-line flags (`conns=1`, `sendfile=0`, `sndbuf=0`). We execute a systematic ablation series comparing the Case 2 baseline against parallel connections, zero-copy `sendfile()`, and buffer tuning under $p \in \{0, 0.2, 0.5\}$.

Table 6: Architecture of the parallel transfer implementation.

Design Dimension	Implementation Strategy
Work Distribution	The client downloads the complete manifest, partitions the file list into contiguous chunks of approximately 1 MiB each (at least one whole file per batch), and enqueues all batches into a thread-safe task queue.
Dynamic Scheduling	N worker threads each maintain an independent TCP socket. Each worker pops the next batch from the queue, transmits `FETCH`, streams the batch, and requests another. Faster links naturally process more batches without requiring static partitioning.
File Reassembly	Every individual file is assigned to exactly one worker and written directly to its dedicated partial file `<name>.part`. When a file finishes, it is renamed to `<name>`, avoiding complex byte-level stitching.
Checkpointing	The checkpoint file `.ckpt` records the session ID, a completed-file bitmask (one bit per file), and active in-flight batch ranges. It is atomically flushed under mutex synchronization after every file and every 64 KiB.
Failure Recovery	When an individual worker socket is severed by a fault, only its active batch is affected. The worker truncates its current file, re-enqueues the remaining batch range at the front of the queue, reconnects via `ATTACH`, and resumes. The remaining $N - 1$ workers continue transferring without interruption.
Termination	Worker threads disconnect when the queue is depleted. The primary connection issues `DONE <sid>` once all files are verified on disk.
Server Session Sharing	Server-side connections share a single reference-counted `session_t`. `FETCH` commands validate index ranges and offsets against topic limits without requiring global high-water checks across non-contiguous batches.

Table 7: Optional performance enhancements and expected system effects.

Enhancement	Implementation Mechanism	Anticipated Performance Effect
Zero-Copy Send	Replaces 64 KiB `read()`/`send()` loop with Linux `sendfile()` directly between disk fd and socket fd.	Eliminates user-space memory copies and context switches; lowers CPU utilization during file streaming.
Header Coalescing	Enables `TCP_CORK` (or `MSG_MORE`) during header write and clears it after file data.	Forces header and initial file bytes into a single MTU frame; eliminates small framing packets.
Socket Buffering	Benchmarks explicit `SO_SNDBUF` / `SO_RCVBUF` against default Linux TCP autotuning.	Linux autotuning dynamically scales up to `tcp_wmem` limits; explicit sizing retained only if empirical gains emerge.
Client Buffering	Configures 1 MiB `setvbuf()` user buffer on disk write fds; defers explicit `fsync()`.	Batches disk I/O system calls on the client receiver.
Persistent Sockets	Multiple consecutive `FETCH` batches reuse established connections.	Amortizes TCP three-way handshake and slow-start ramp-up across multiple batches.

7 Additional Features Beyond the Minimum

Table 8 highlights supplementary architectural features integrated into the service to ensure stability, debugging visibility, and experimental rigor without modifying the mandatory CLI signature.

Table 8: Additional system features beyond minimum project specifications.

Feature	Implementation Detail	Design Rationale
Configurable Parameters	Trailing `key=value` flags: `mode`, `seed`, `ckpt_kb`, `conns`, `max_clients`, `max_retries`, `session_ttl`, `fault_mode`, `sendfile`, `sndbuf`, `log`.	Enables experimental tuning while remaining 100% compatible with auto-grader invocations.
Per-Connection CSV Metrics	Server and client append structured metrics rows recording timestamps, bytes sent/received, payload, and status.	Provides raw data logging for offline throughput, goodput, and overhead analysis in the `Results/` directory.
Leveled Logging	Leveled logging (`log=0..3`, error to debug) to `stderr`. Background logger thread writes CSVs in multi-threaded mode.	Keeps disk write latencies completely off the real-time network transfer path.
Graceful Shutdown	`SIGINT` / `SIGTERM` captured via a self-pipe; closes listening socket, allows 5 s for active transfers to drain, and flushes CSV logs.	Prevents socket leaks, hanging ports, or truncated metric logs when stopping test runs.
Network Robustness	Handles `SIGPIPE` via `SIG_IGN`, uses `MSG_NOSIGNAL`, enforces 10 s read timeout on requests, retries on `EINTR`.	Ensures a misbehaving or stalled client cannot hang server threads indefinitely.
Path Traversal Safety	Rejects topic requests containing `/` or `..`; client enforces strict basename extraction for local writes.	Prevents arbitrary file reads from server or malicious directory writes on client.
Dynamic Extensibility	Scans topic root directory at startup; new topics are registered automatically by adding directories.	Eliminates code modifications when adding new datasets or topics.
LIST Command	Diagnostic command returning topic keys, file counts, and sizes.	Facilitates rapid manual verification via `nc`.
Connection Throttling	Configurable `max_clients` bound (default 128).	Protects the server against thread exhaustion during massive concurrent client sweeps.
End-to-End Integrity	`verify.sh` script verifies SHA-256 digests of all received files against original server files.	Confirms that fault recovery and checkpoint resumption never corrupt or drop bytes.
Deterministic Exit Codes	Standardized codes: `0` (success), `1` (usage error), `2` (retry limit reached), `3` (topic not found), `4` (I/O error).	Allows shell test scripts to detect and handle error conditions programmatically.

8 Code Structure

The submission archive is structured into modular, self-contained directories. Each variant folder contains its own `Makefile` and builds into exactly two binaries: `server` and `client`.

```
FinalSubmission_Group_021_034/
|-- FinalReport_Group_021_034.pdf
|-- README.md
|-- Part1/
```

```
|   |-- Makefile, common.[ch], topics.[ch], proto.[ch], metrics.[ch]
|   `-- server.c, transfer.c, client.c
|-- Part2_Case1/
|   `-- [Part1 files] + fault.[ch], retry loop in client.c
|-- Part2_Case2/
|   `-- [Case1 files] + session.[ch], checkpoint.[ch]
|-- Part2_Case2_Enhanced/
|   `-- [Case2 files] + manifest.c, workq.[ch], parallel client
|-- Results/
|   |-- Part1_SingleThread_throughput.[csv|png], Part1_MultiThread_throughput.[csv|
    png]
|   `-- Part2_Case1_throughput.png, Part2_Case1_overhead.png ...
`-- scripts/ (run_part1.sh, run_part2.sh, verify.sh, plot.py)
```

Listing 3: Repository directory layout.

8.1 Key Modules and Core Functions

Table 9 defines the primary software modules and their functional roles across the codebase.

Table 9: Code modules and key functions.

Module	Key Functions	Usage Scope
common.c	send_all(), lr_init(), lr_readline(), lr_read_exact(), now_ns(), parse_opts()	Shared across all servers and clients.
topics.c	topic_map_build(), topic_lookup(), topic_pos(), topic_stream_size()	Topic scanning and indexing; all servers.
proto.c	proto_parse_request(), proto_fmt_file_hdr(), proto_parse_file_hdr()	Control line parsing and serialization; all modules.
transfer.c	send_topic(), send_file_range()	File reading and socket streaming; all servers.
server.c	main(), listen_on(), accept_loop(), serve_client(), client_thread()	Server lifecycle and socket dispatch; all servers.
metrics.c	metrics_row(), background logger thread	Asynchronous performance row emission; all modules.
fault.c	fault_plan_new(), fault_budget(), fault_abort()	Fault plan evaluation and socket reset; Part II servers.
session.c	session_create(), session_validate_resume(), session_update_hw(), reaper_thread()	Session table lifecycle and validation; Case 2 & Enhanced.
checkpoint.c	ckpt_load(), ckpt_save_atomic(), ckpt_remove()	Client disk checkpoint persistence; Case 2 & Enhanced.
client.c	fetch_topic(), recv_files(), retry_loop(), worker_main()	Client connection, download, and retry logic.
workq.c	workq_build(), workq_take(), workq_requeue()	Thread-safe parallel batch queue; Enhanced client.

8.2 Variant Composition and Layering

Table 10 summarizes how each progressive variant extends the core codebase.

8.3 Build System and Command-Line Interface

Compilation enforces strict compliance flags:

```
gcc -std=c11 -O2 -Wall -Wextra -Werror -pthread
```

Table 10: Architectural differentiation across project variants.

Project Variant	Implementation Layering and Differences
Part I: Single vs. Multi	Both share the identical `serve_client()` routine. In single-threaded mode, the accept loop invokes it sequentially while pending clients queue in the listen backlog (128). In multi-threaded mode, each client triggers a detached thread running `client_thread()`. Configured via `mode=single|multi` (default `multi`).
Part II: Case 1	Integrates `fault.c`. Server passes a fault plan into `send_topic()`, capping byte writes via `fault_budget()` and calling `fault_abort()` upon budget depletion. Client wraps `fetch_topic()` in a backoff retry loop.
Part II: Case 2	Adds a `RESUME` handling branch in `serve_client()` calling `session_validate_resume()` and resuming `send_topic()` from the verified checkpoint. Client logs checkpoints inside `recv_files()`.
Part II: Enhanced	Adds `ATTACH` and `FETCH` handling loops in server. Client replaces single receive loop with N concurrent worker threads pulling batches from `workq`.

Every source unit defines `_GNU_SOURCE` before header inclusion to expose POSIX extensions (`erand48()`, `strcasecmp()`, and `sendfile()`) cleanly. The CLI conforms strictly to assignment guidelines:

Table 11: Command-line interface specifications.

Program Target	Mandatory Invocations	Supported Optional Arguments
Part I Server	`./server <port> <topics_root_dir>`	`mode=`, `log=`, `max_clients=`
Part II Server (All Cases)	`./server <port> <topics_root_dir> <failure_probability>`	`seed=`, `fault_mode=`, `session_ttl=`, `log=`, `max_clients=`; Enhanced: `sendfile=`, `sndbuf=`
Client (All Cases)	`./client <server_ip> <server_port> <topic> <output_dir>`	`max_retries=`, `ckpt_kb=`, `conns=`, `log=`

9 Design Thought Process and Alternatives

Our primary design philosophy was to introduce exactly one core mechanism per case, maintain identical supporting structures across all variants, and select predictable algorithms whose behavior can be mathematically verified.

9.1 Architectural Synthesis by Case

9.1.1 Part I: Threading Trade-offs

A thread pool avoids per-connection creation overhead. However, on Linux systems, creating a thread requires approximately 15 μs, whereas streaming a multi-megabyte topic requires hundreds of milliseconds. We expect thread-per-client to have low overhead for the expected workload, while simplifying connection handling. We expect the single-threaded server to serialize client transfers, while the multi-threaded server can overlap transfers across clients.

9.1.2 Part II Case 1: Defining Fault Probability

Evaluating fault injection on a per-chunk basis makes completing large files statistically impossible even under low error rates ($1 - q^{1600} \approx 0$). Defining p as a per-request trial ensures the transfer is provably solvable, maintains a direct mapping between p and the expected number of retry attempts ($\frac{1}{1-p}$), and enables rigorous validation against empirical measurements.

Table 12: Architectural decisions and evaluated alternatives.

Decision Area	Alternatives Evaluated	Selected Approach	Design Rationale
Wire Protocol	Fixed binary structs in network byte order; TLV framing.	ASCII text headers with raw binary payload.	Complies with `GET <topic>` brief, transparent debugging via `nc`, header overhead $< 0.1\%$.
Topic Map	Dynamic hash table; disk rescanning per request.	Sorted in-memory arrays via `bsearch()`.	Exactly 10 topics; enables stable indexing for checkpoints and lock-free concurrent reads.
Concurrency Model	Pre-allocated thread pool with task queue.	Thread-per-client (detached) capped by `max_clients`.	Minimal overhead up to 32 clients; threads cost microseconds while file transfers last seconds.
Task Threading	Monolithic thread per client doing all tasks.	Accept and worker threads; async logger and reaper.	Isolates disk write latencies and session garbage collection from network streaming.
Fault Injection	Per-chunk Bernoulli trial; timer-based abort.	One Bernoulli trial per request; uniform cut $K \sim \mathcal{U}[0, R-1]$.	Prevents infinite restart loops in Case 1; yields closed-form mathematical expectations.
Enhanced Fault Unit	One Bernoulli trial per `FETCH` batch.	One Bernoulli trial per TCP connection.	Ensures equal fault frequency between Case 2 and Case 2 Enhanced at identical p.
Fault Mechanism	Graceful `close()` (`TCP FIN`); `shutdown()`.	Ungraceful reset via `SO_LINGER 0` `close()`.	Transmits `TCP RST`, accurately mimicking real host crashes and abrupt link drops.
Case 1 Restart	Retain finished files; resume incomplete files.	Purge partial attempt files; restart from byte 0.	Complies strictly with brief mandate: no retained transfer metadata in Case 1.
Checkpoint Authority	Server-side tracking via `send()` count; ACKs.	Client disk checkpoint validated by server high-water.	Only client knows what reached disk; eliminates upstream ACK traffic.
Resume Comparison	Compare $(\text{index}, \text{offset})$ tuples directly.	Compare continuous scalar byte positions.	File boundary ambiguity $((\text{file}_i, \text{size}_i) \equiv (\text{file}_{i+1}, 0))$ cleanly resolved.
Parallel Splitting	Static round-robin file split; static byte chunks.	Client-driven queue of $\sim$ 1 MiB batches.	Automatic load balancing across asymmetric links; failed connection stalls only one batch.
Socket Mode	Non-blocking sockets with `epoll` event loop.	Blocking sockets with POSIX worker threads.	Straightforward linear error handling during faults; perfectly suited for tens of connections.

9.1.3 Part II Case 2: Checkpoint Integrity and Validation

Relying on server-side write accounting is flawed because `send()` simply deposits data into local kernel buffers; an ungraceful reset flushes unsent buffer contents without client delivery. Requiring client ACK packets would double control traffic and require bidirectional worker logic. Establishing the client's local disk state as the authority, checked against an uncontended server high-water mark, provides complete correctness with minimal overhead. Furthermore, advancing `hw_pos` after every socket write ensures that intra-file checkpoints within large images are recognized and honored upon reconnection.

9.1.4 Part II Case 2 Enhanced: Batch Granularity and Load Balancing

Static file allocations perform poorly when a single connection experiences repeated faults and stalls overall completion. Dynamic batches of $\sim 1\,\text{MiB}$ allow healthy connections to rapidly drain the work queue while a severed link recovers. This batch size strikes an optimal balance between scheduling overhead (round-trip `FETCH` frames) and the volume of work tied to an individual connection.

References

[1] A. Corrado, "Animals-10 Dataset," *Kaggle*, 2020. [Online]. Available: https://www.kaggle.com/datasets/alessiocorrado99/animals10

[2] IEEE and The Open Group, "POSIX.1-2024 / IEEE Std 1003.1-2024 Base Specifications, Issue 8," 2024. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9799919799/

[3] Lawrence Livermore National Laboratory (LLNL), "POSIX Threads Programming Tutorial," High Performance Computing Training. [Online]. Available: https://hpc-tutorials.llnl.gov/posix/

[4] Michael Kerrisk et al., *Linux Programmer's Manual*, pages: `socket(7)`, `tcp(7)`, `sendfile(2)`, `clock_gettime(2)`.